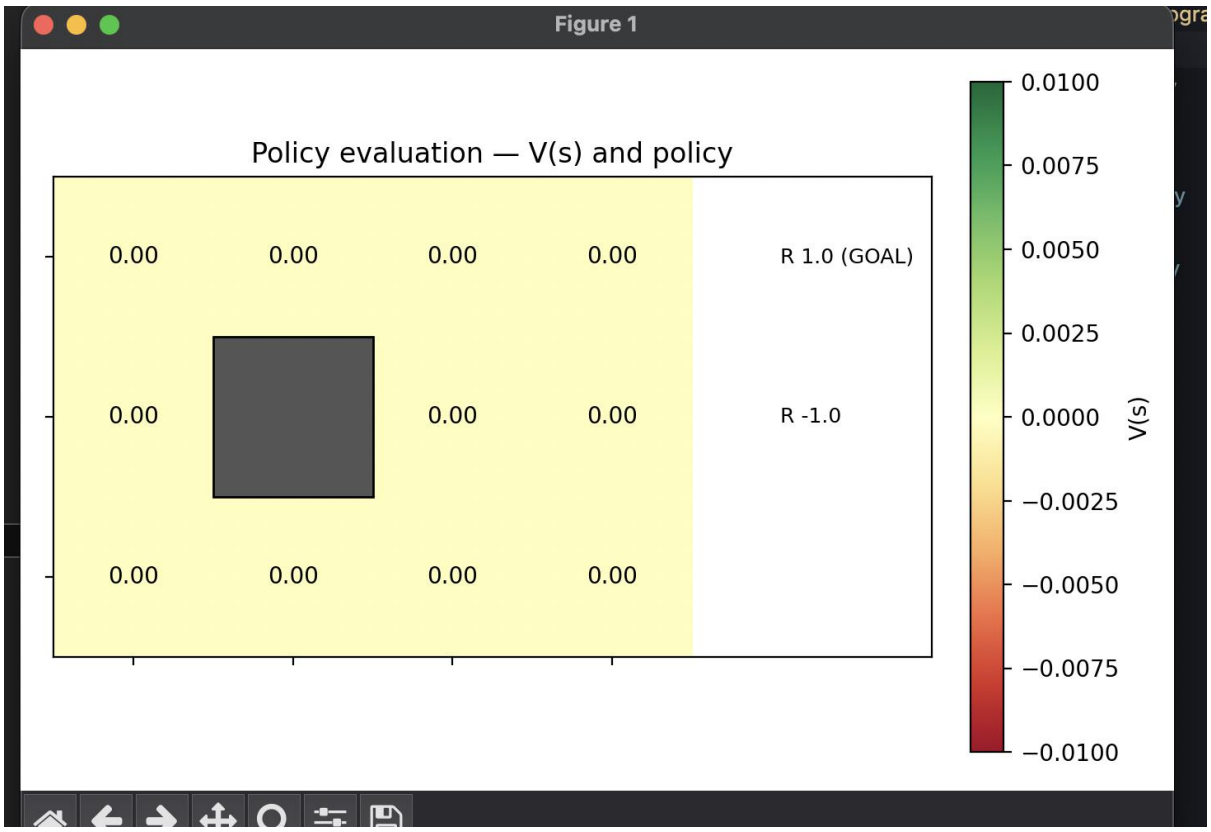
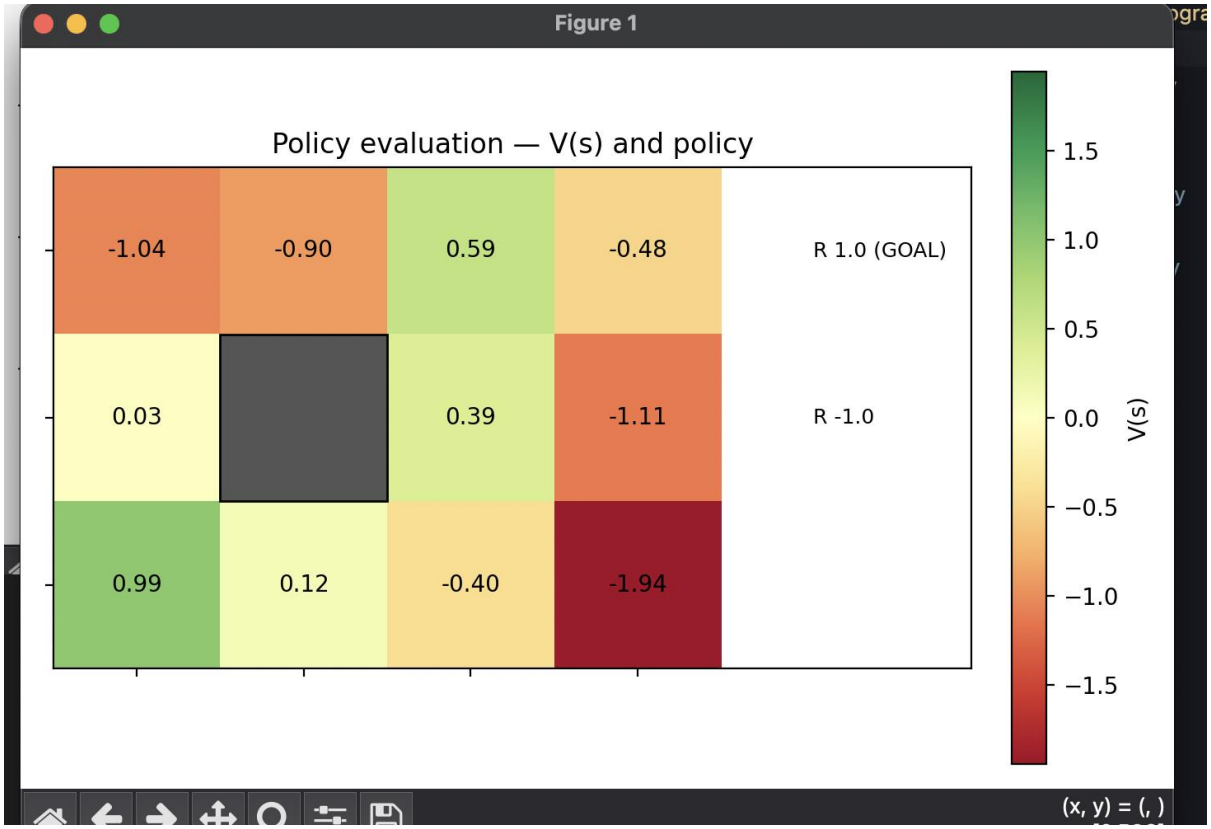
```
"""
실습 #1 gridworld.py
"""

import numpy as np
import common.gridworld_render as render_helper



class GridWorld:
def __init__(self):
self.action_space = [0, 1, 2, 3] # 행동 공간(가능한 행동들)
self.action_meaning = { # 행동의 의미
0: "UP",
1: "DOWN",
2: "LEFT",
3: "RIGHT",
}

self.reward_map = np.array( # 보상 맵(각 좌표의 보상 값)
[[0, 0, 0, 1.0],
[0, None, 0, -1.0],
[0, 0, 0, 0]]
)

self.goal_state = (0, 3) # 목표 상태(좌표)
self.wall_state = (1, 1) # 벽 상태(좌표)
self.start_state = (2, 0) # 시작 상태(좌표)
self.agent_state = self.start_state # 에이전트 초기 상태(좌표)

self.terminal_states = set()
for h in range(len(self.reward_map)):
for w in range(len(self.reward_map[0])):
r = self.reward_map[h, w]
if r is not None and r != 0:
self.terminal_states.add((h, w))

@property
def height(self):
return len(self.reward_map)

@property
def width(self):
return len(self.reward_map[0])
```

```
@property
def shape(self):
return self.reward_map.shape

def actions(self):
return self.action_space

def states(self):
for h in range(self.height):
for w in range(self.width):
yield (h, w)

def next_state(self, state, action):
action_move_map = [(-1, 0), (1, 0), (0, -1), (0, 1)]
move = action_move_map[action]
next_state = (state[0] + move[0], state[1] + move[1])
ny, nx = next_state

if nx < 0 or nx >= self.width or ny < 0 or ny >= self.height:
next_state = state
elif next_state == self.wall_state:
next_state = state

return next_state

def reward(self, state, action, next_state):
r = self.reward_map[next_state]
if r is None:
return 0.0
return float(r)

def reset(self):
self.agent_state = self.start_state
return self.agent_state

def step(self, action):
state = self.agent_state
next_state = self.next_state(state, action)
reward = self.reward(state, action, next_state)
done = next_state in self.terminal_states

self.agent_state = next_state
```

```
return next_state, reward, done

def render_v(self, v=None, policy=None, print_value=True, use_matplotlib=True):
renderer = render_helper.Renderer(
self.reward_map, self.goal_state, self.wall_state
)
renderer.render_v(v, policy, print_value, use_matplotlib=use_matplotlib)

def render_q(self, q=None, print_value=True):
renderer = render_helper.Renderer(
self.reward_map, self.goal_state, self.wall_state
)
renderer.render_q(q, print_value)



if __name__ == "__main__":
env = GridWorld()

print(env.height)
print(env.width)
print(env.shape)

for action in env.actions():
print(action)

print("===")

for state in env.states():
print(state)

print("\n--- step / render_v ---")
print("goal", env.goal_state, "| wall", env.wall_state, "| start", env.start_state)
print("reset ->", env.reset())
s, r, d = env.step(3)
print("step RIGHT(3) ->", s, "r=", r, "done=", d)

print("빈 가치로 그리드(0 / G 목표 / ### 벽):")
env.render_v(use_matplotlib=False)

V = {}
for state in env.states():
V[state] = np.random.randn()
```

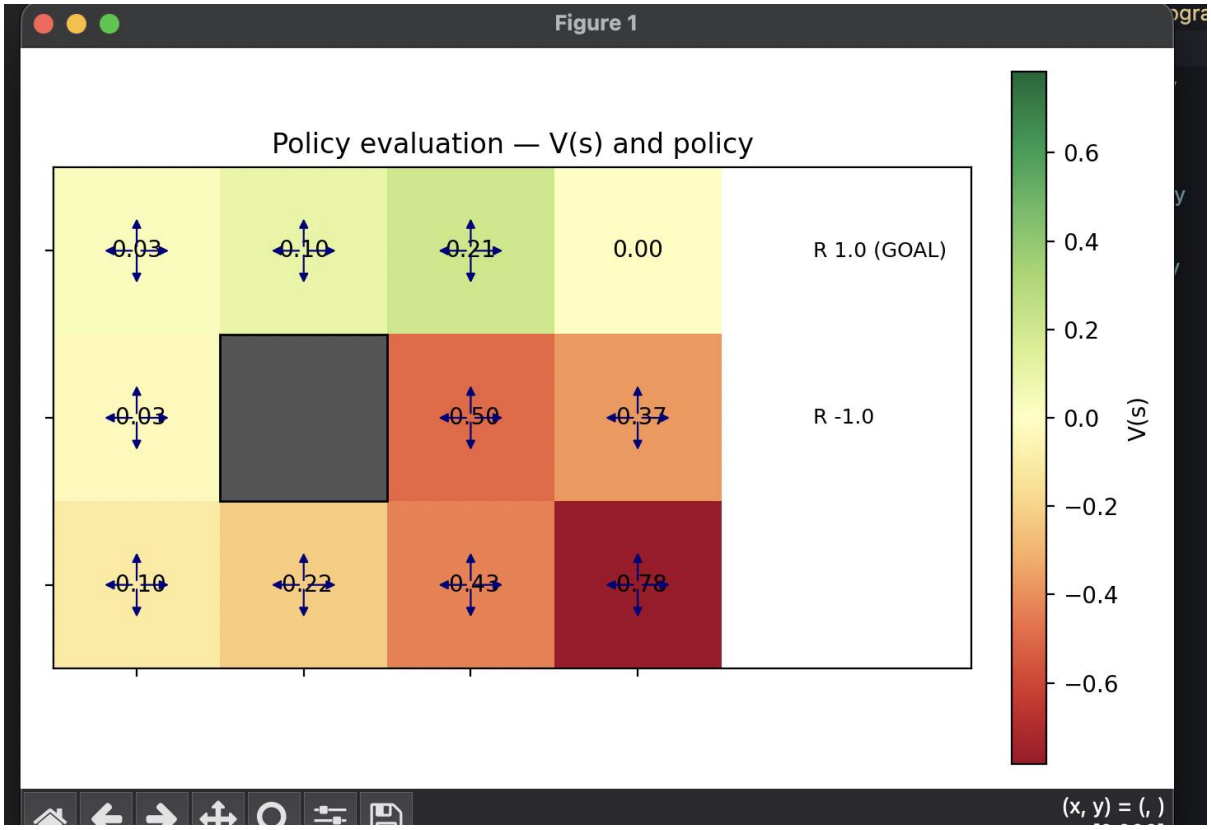
```
env.render_v(V, use_matplotlib=False)
```





```
# 실습 #1 policy_eval.py

from collections import defaultdict
from gridworld import GridWorld



def eval_onestep(pi, V, env, gamma=0.9):
for state in env.states(): # 각 상태에 접근
if state == env.goal_state: # 2 목표 상태에서의 가치 함수는 항상 0
V[state] = 0
continue

action_probs = pi[state]
new_V = 0

# 각 행동에 접근
for action, action_prob in action_probs.items():
next_state = env.next_state(state, action)
r = env.reward(state, action, next_state)
# 새로운 가치 함수
new_V += action_prob * (r + gamma * V[next_state])

V[state] = new_V
return V



def policy_eval(pi, V, env, gamma, threshold=0.001):
while True:
old_V = V.copy() # 갱신 전 가치 함수
V = eval_onestep(pi, V, env, gamma)

# 갱신된 양의 최댓값 계산
delta = 0
for state in V.keys():
t = abs(V[state] - old_V[state])
if delta < t:
delta = t
```

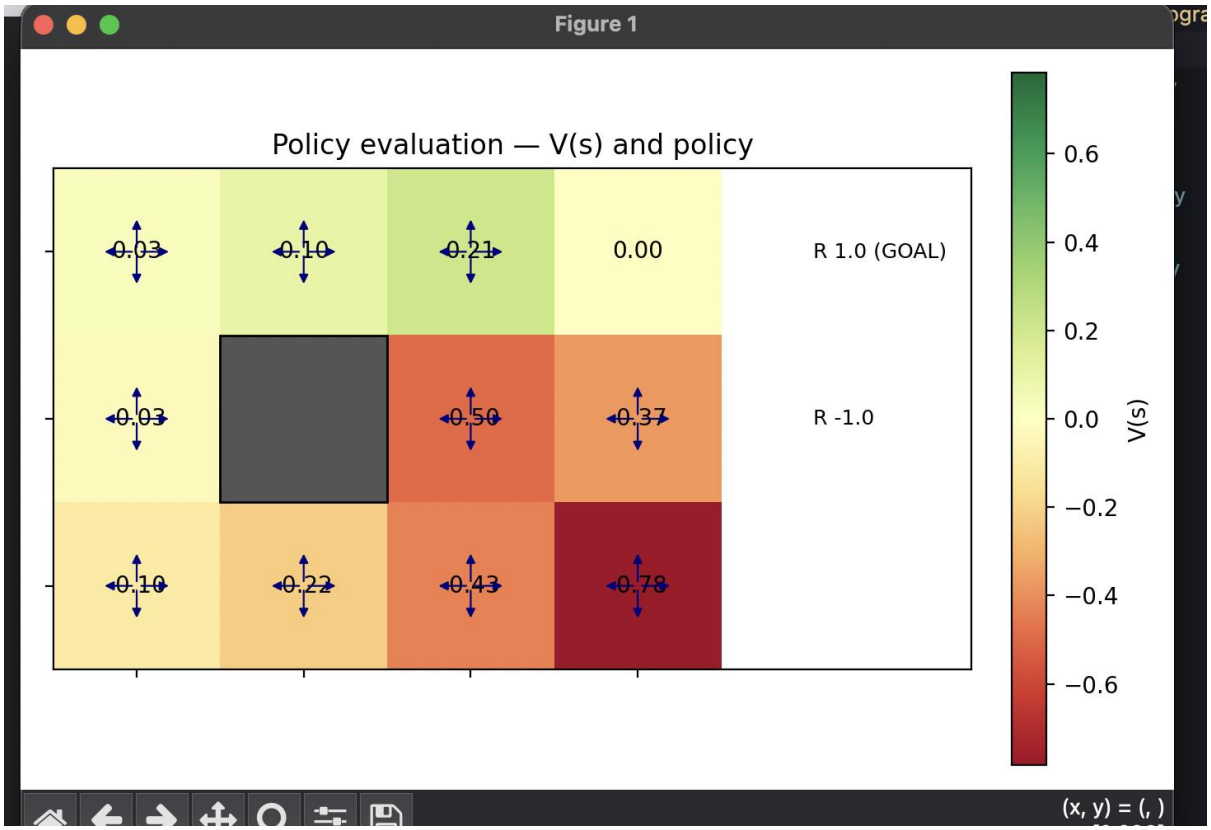
```
# 임계값과 비교
if delta < threshold:
break
return V



if __name__ == "__main__":
env = GridWorld()
gamma = 0.9 # 할인율

pi = defaultdict(lambda: {0: 0.25, 1: 0.25, 2: 0.25, 3: 0.25}) # 정책
V = defaultdict(lambda: 0) # 가치 함수

V = policy_eval(pi, V, env, gamma) # 정책 평가

# [그림 4-13] 무작위 정책의 가치 함수
env.render_v(V, pi)
```



```python
# 실습 #2 policy_iter.py


from collections import defaultdict


from common.gridworld import GridWorld
from policy_eval import policy_eval



def argmax(d):
"""d (dict)"""
max_value = max(d.values())
max_key = -1
for key, value in d.items():
if value == max_value:
max_key = key
return max_key
```

```
def greedy_policy(V, env, gamma):
pi = {}

for state in env.states():
action_values = {}

for action in env.actions():
next_state = env.next_state(state, action)
r = env.reward(state, action, next_state)
value = r + gamma * V[next_state]
action_values[action] = value

max_action = argmax(action_values)
action_probs = {0: 0, 1: 0, 2: 0, 3: 0}
action_probs[max_action] = 1.0
pi[state] = action_probs
return pi



def policy_iter(env, gamma, threshold=0.001, is_render=True):
pi = defaultdict(lambda: {0: 0.25, 1: 0.25, 2: 0.25, 3: 0.25})
V = defaultdict(lambda: 0)

while True:
V = policy_eval(pi, V, env, gamma, threshold) # 평가
new_pi = greedy_policy(V, env, gamma) # 개선

if is_render:
env.render_v(V, pi)

if new_pi == pi: # 갱신 여부 확인
break
pi = new_pi

return pi



# 가치 함수와 정책 시각화(책의 [그림 4-16]은 처음과 마지막 그래프만 표시했음)
if __name__ == "__main__":
```

```
env = GridWorld()
gamma = 0.9


pi = policy_iter(env, gamma)
```



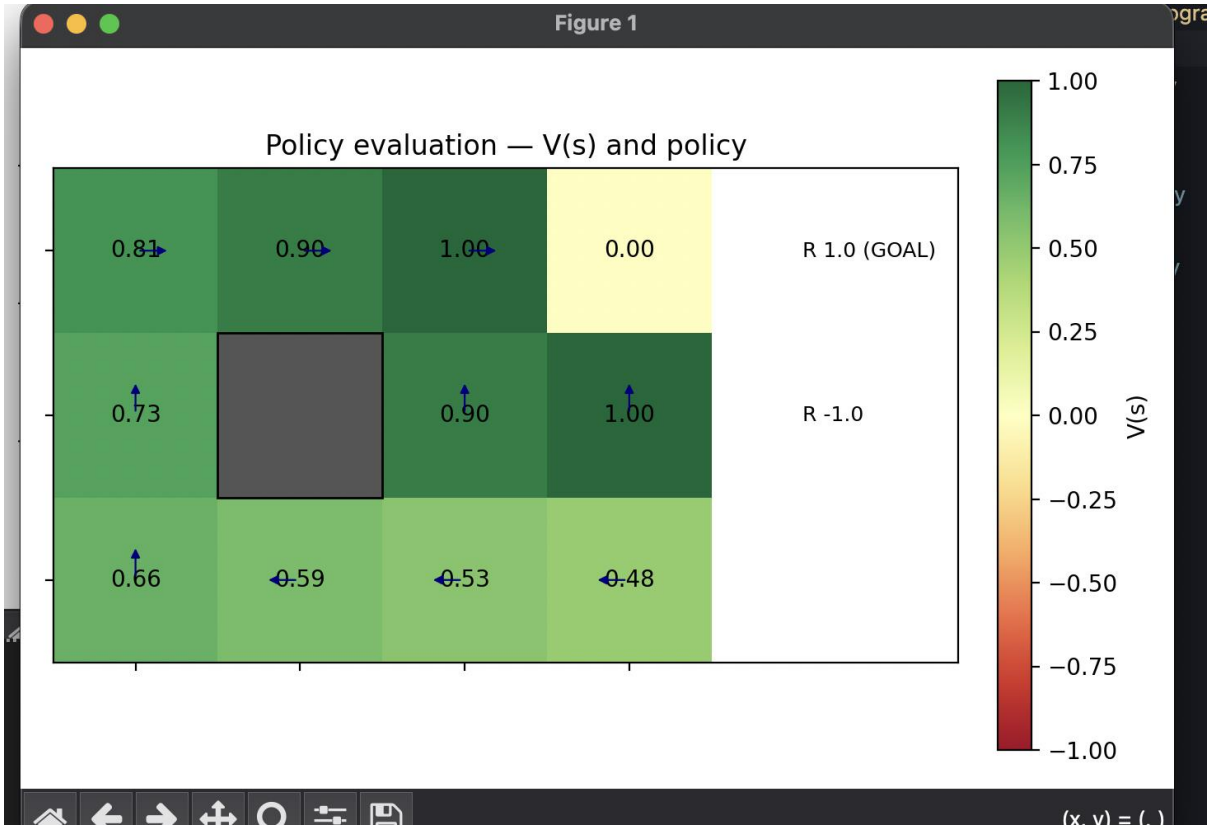


Figure 1
Policy evaluation — V(s) and policy
0.81
0.90
1.00
0.00
R 1.0 (GOAL)
0.73
0.90
1.00
R -1.0
0.66
0.59
0.53
0.48
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
V(s)

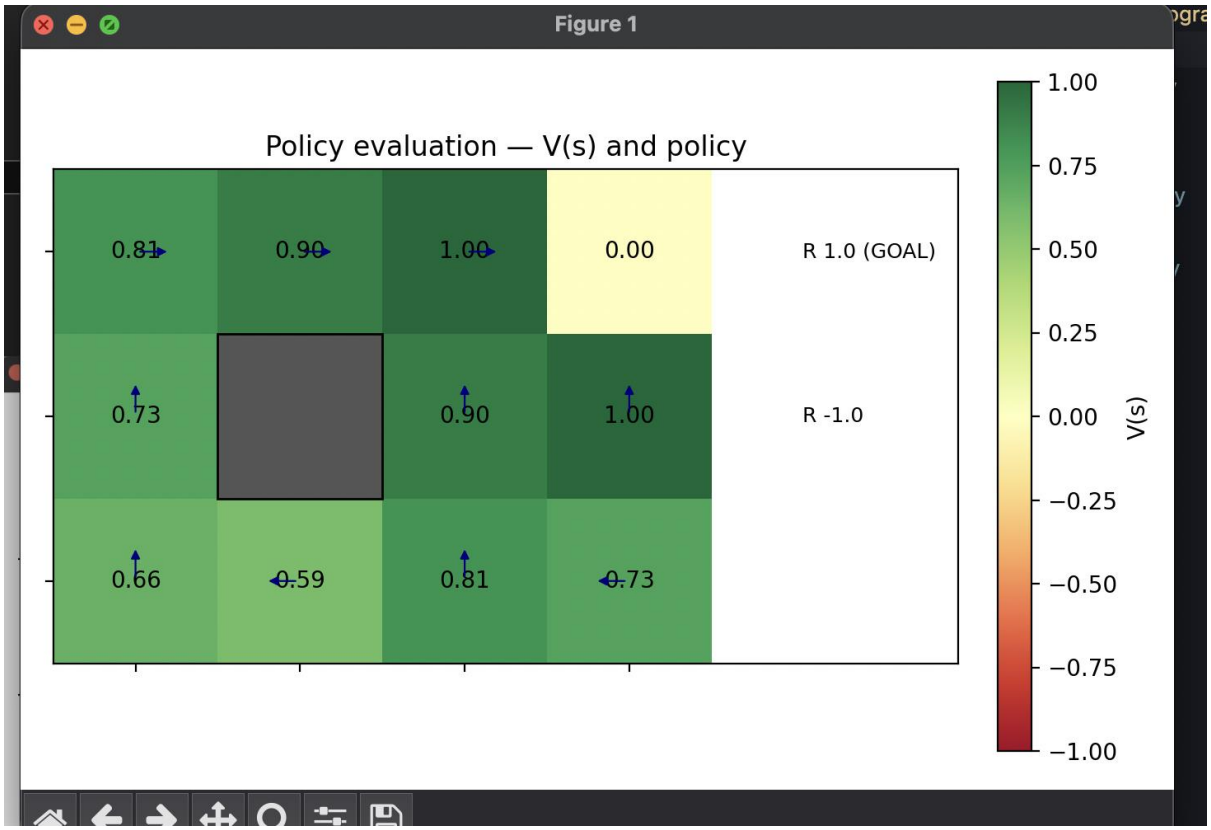


Figure 1
Policy evaluation — V(s) and policy
0.81
0.90
1.00
0.00
R 1.0 (GOAL)
0.73
0.90
1.00
R -1.0
0.66
0.59
0.81
0.73
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
V(s)

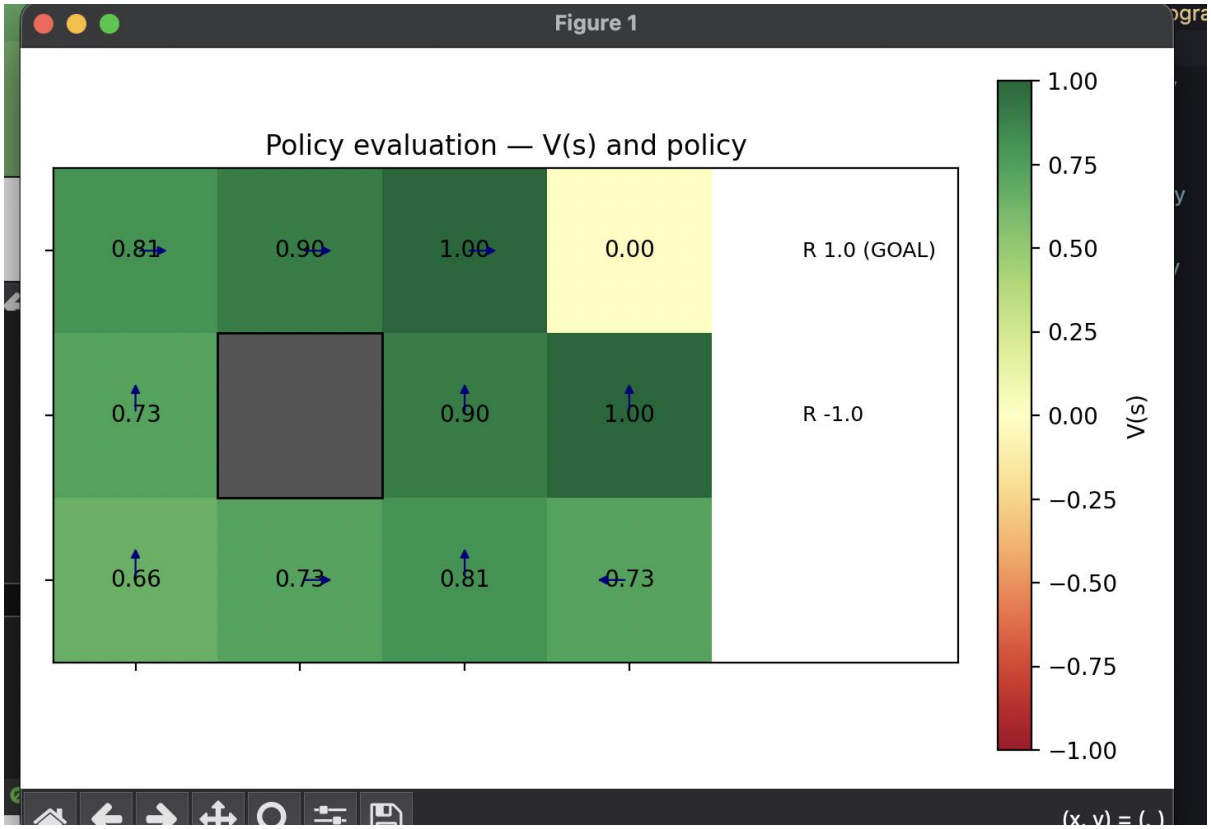


Figure 1
Policy evaluation — V(s) and policy
0.81
0.90
1.00
0.00
R 1.0 (GOAL)
0.73
0.90
1.00
R -1.0
0.66
0.73
0.81
0.73
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
V(s)

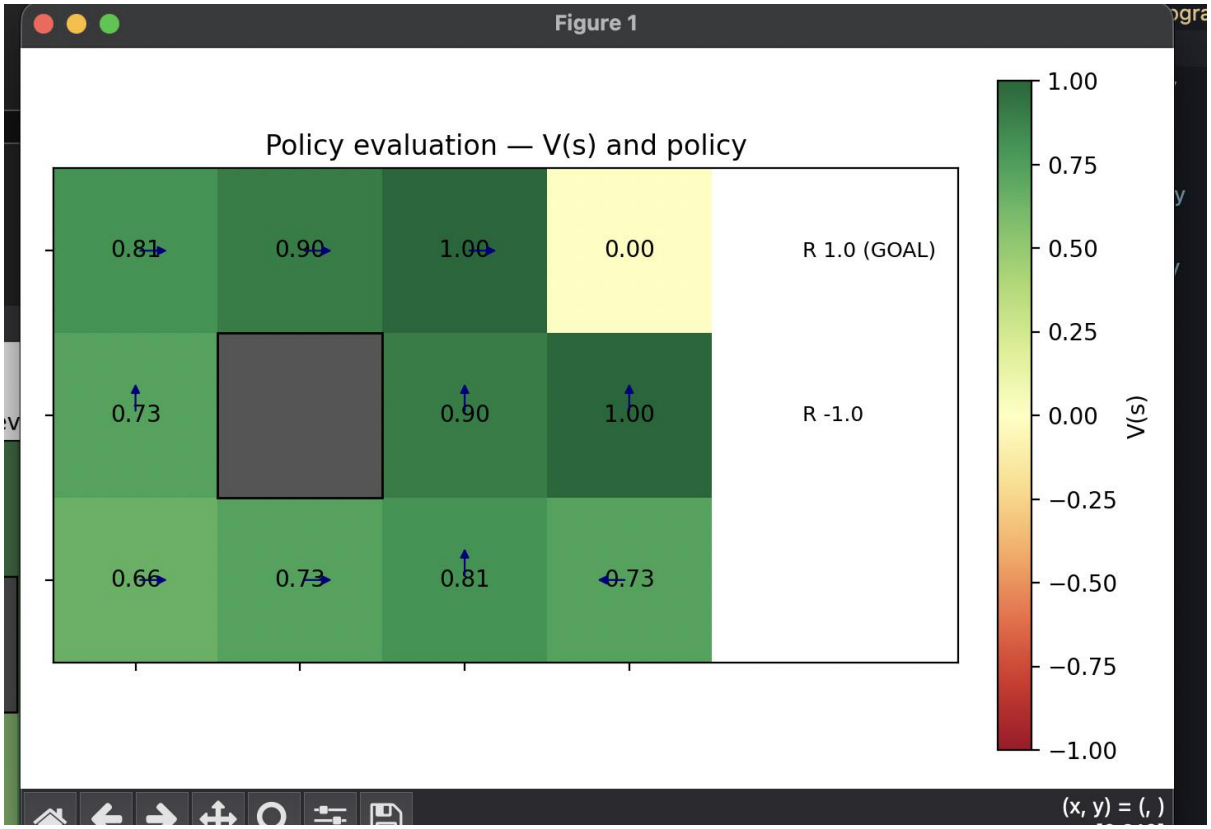


Figure 1
Policy evaluation — V(s) and policy
0.81
0.90
1.00
0.00
R 1.0 (GOAL)
0.73
0.90
1.00
R -1.0
0.66
0.73
0.81
0.73
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
V(s)
(x, y) = (,)